Gas Optimization Report

`CrowdFund.sol` Smart Contract

Contents

Summary Finding: The `createCampaign()` function contains three distinct storage inefficiencies that together waste approximately **~60,000–80,000 gas** per call, representing the single largest per-transaction gas cost in the contract.

1. Background and Scope

The following report focus excluisvely on the gas optimization proposed for the Crowd Funding smart contract. The contract exposes four user-facing functions: `createCampaign()`, `contribute()`, `withdraw()`, and `refund()`.

1.1. EVM Storage Cost Primer

Understanding the findings requires a brief review of EVM storage opcodes:

Table 1: Key EVM storage opcode costs (post-Berlin, EIP-2929)

Opcode	Condition	Gas	When triggered
`SSTORE`	Zero → non-zero (cold)	22,100	First write to a new slot
`SSTORE`	Non-zero → non-zero	5,000	Update an existing slot
`SSTORE`	Non-zero → zero	5,000 + 4,800 refund	Clearing a slot
`SLOAD`	Cold access	2,100	First read of a slot
`SLOAD`	Warm access	100	Repeated read within tx
`MSTORE` (memory write)		3	In-memory, per 32 bytes

The fundamental rule we followed is: **every new storage slot costs ∼22,100 gas to initialise**. Removing unnecessary storage writes is therefore the highest-leverage optimization available for the contract.

2. Most Gas-Intensive Function: `createCampaign()`

The most usage in the contract is done by the `createCampaign()` function which deals with creating and storing the campaign , with the goal, deadline and other attributes.

2.1. Original Implementation

The original contract implementation stores the relevant attributes of a campaign in a `struct` structure. Every `createcampaign()` fucntion call writes into this cold storage opting to high gas usage, hence optimizing the `struct` meant optimizing the `createcampaign()` function.

```
struct campaign {
    address owner;
    status  state;
    uint    campaignid;      // <-- redundant field
    uint    goalWei;
    uint    deadlineTimestamp;
    uint    amountcontributed;
    string  metadataCID;     // <-- dynamic string in storage
    mapping(address => uint) contributions;
}

uint[] public campaign_id_list;          // <-- dynamic array pushed
    on every create
mapping(uint => campaign) public campaign_map;
uint256 public campaignCount;
```

```
15
16 function createCampaign(
17     uint256 _goalWei,
18     uint256 _deadlineTimestamp,
19     string calldata _metadataCID
20 ) external returns (uint) {
21     require(_goalWei > 0, "Funds requested invalid");
22     require(_deadlineTimestamp > block.timestamp, "Deadline invalid")
           ;
23
24     uint256 campaign_id = campaignCount++;
25
26     campaign_id_list.push(campaign_id);          // SSTORE x2 (slot +
           length)
27
28     campaign storage newCampaign = campaign_map[campaign_id];
29     newCampaign.campaignid        = campaign_id; // SSTORE #1 --
           redundant
30     newCampaign.owner             = msg.sender;  // SSTORE #2
31     newCampaign.goalWei           = _goalWei;    // SSTORE #3
32     newCampaign.deadlineTimestamp = _deadlineTimestamp; // SSTORE #4
33     newCampaign.amountcontributed = 0;           // no-op (default),
           still compiled
34     newCampaign.state             = status.Active; // packed with
           owner slot
35     newCampaign.metadataCID       = _metadataCID; // SSTORE x2-3 (len
           + data chunks)
36
37     emit CampaignCreated(...);
38     return campaign_id;
39 }
```

Listing 1: Original `createCampaign()` – `crowdfund_main.sol`

2.2. Gas Breakdown of Original `createCampaign()`

Table 2: Storage writes (SSTOREs) in original `createCampaign()`

#	Operation	Slots written	Est. gas
1	`campaign_id_list.push()` — new element slot	1	~22,100
	`campaign_id_list.push()` — array length update	1	~5,000
2	`newCampaign.campaignid = campaign_id`	1	~22,100
3	`newCampaign.owner = msg.sender` (packed with `state` enum in same slot)	1	~22,100
4	`newCampaign.goalWei = _goalWei`	1	~22,100
5	`newCampaign.deadlineTimestamp = ...`	1	~22,100
6	`newCampaign.metadataCID` — length slot	1	~22,100
	`newCampaign.metadataCID` — data (46–59 byte CID)	2	~44,200
	Base tx overhead + other opcodes	—	~21,000
	Total (estimated)	**9–10**	**~200,000**

Finding: `createCampaign()` performs **9–10 cold `SSTORE` operations** in a single transaction, making it by far the most gas-expensive function in the contract. Three of these storage writes are entirely avoidable.

3. Identified Inefficiencies

3.1. Inefficiency 1 — Redundant `campaignid` Field in Struct

Description

The `campaign` struct stores `uint campaignid`, which is written on every `createCampaign()` call taking up one full storage:

```
newCampaign.campaignid = campaign_id;  // one full SSTORE
```

Campaigns can be accessed through *key* of `campaign_map` by any caller directly instead of storing it in the `struct`.

Gas cost

Initialising a new 32-byte storage slot from zero to a non-zero value costs **22,100 gas** (EIP-2929, cold `SSTORE`). Spending that much gas on every function call is not ideal and unneccesary.

Fix

Remove `uint campaignid` from the struct entirely. The field is never read inside the contract itself.

```
// BEFORE
struct campaign {
    address owner;
    status  state;
    uint    campaignid;       // redundant -- remove this line
    uint    goalWei;
    ...
}

// AFTER
struct campaign {
    address owner;
    status  state;
    uint    goalWei;          // campaignid field gone
    ...
}
```

Listing 2: Before (issueBox) vs. After (fixBox)

Saving: ~22,100 gas per `createCampaign()` call.

3.2. Inefficiency 2 — Dynamic `string metadataCID` in Storage

Description

The struct field `string metadataCID` persists the IPFS CID on-chain:

```
newCampaign.metadataCID = _metadataCID;
```

Solidity stores dynamic strings in two parts:

1. A **length slot** (32 bytes) at the struct's storage position.
2. **Data chunks** of 32 bytes each at a hash-derived location.

An IPFS CIDv1 (e.g. `Qm...`) is 46–59 bytes, requiring **2 data slots** in addition to the length slot — three cold `SSTORE` operations in total.

Gas cost

$$3 \times 22{,}100 = \mathbf{66{,}300} \quad \text{gas}$$

Why storage is unnecessary

The `CampaignCreated` event already broadcasts `metadataCID` to the chain's log trie:

```
emit CampaignCreated(campaign_id, msg.sender, _goalWei,
                     _deadlineTimestamp, _metadataCID);
```

Event logs are **permanently accessible** to any off-chain indexer, frontend, or The Graph subgraph. Storing the CID in contract storage as well doubles the cost with no added on-chain utility.

Fix

Remove `string metadataCID` from the struct and from `getCampaign()`. Keep `_metadataCID` as a `calldata` parameter so it flows into the event at negligible cost.

```
// AFTER -- metadataCID no longer in struct
struct campaign {
    address owner;
    status  state;
    uint    goalWei;
    uint    deadlineTimestamp;
    uint    amountcontributed;
    mapping(address => uint) contributions;
}

// getCampaign returns 5 values instead of 6
function getCampaign(uint _campaignId) external view
    returns (uint goal, uint raised, uint deadline,
             address creator, status state)
{
    campaign storage c = campaign_map[_campaignId];
    require(c.owner != address(0), "Campaign does not exist");
    return (c.goalWei, c.amountcontributed,
            c.deadlineTimestamp, c.owner, c.state);
}
```

Listing 3: Optimized struct and getCampaign return signature

Saving: ~66,300 gas per `createCampaign()` call (3 cold `SSTORE` operations eliminated).

3.3. Inefficiency 3 — Unnecessary `campaign_id_list` Array

Description

A dynamic `uint[]` is maintained in storage and pushed to on every campaign creation:

```
uint[] public campaign_id_list;
...
campaign_id_list.push(campaign_id);  // inside createCampaign()
```

A `push()` to a dynamic array triggers two storage writes:

1. Writing the new element to a new slot — ~22,100 gas (cold).
2. Updating the array's length slot — ~5,000 gas (warm, already accessed).

Gas cost

$$22{,}100 + 5{,}000 = \textbf{27,100 gas per campaign}$$

As the list grows, querying `getAllCampaignIds()` also becomes increasingly expensive for callers because each element requires a cold `SLOAD`.

Why the array is unnecessary

Campaign IDs are *sequential integers* assigned by `campaignCount++`, so the full ID space is always $\{0, 1, 2, \ldots, \texttt{campaignCount}-1\}$. This means any caller can reconstruct the list perfectly without any storage array. `getAllCampaignIds()` can build the array in **memory** at query time:

```
// AFTER -- no campaign_id_list state variable
function getAllCampaignIds() external view returns (uint[] memory) {
    uint count = campaignCount;
    uint[] memory ids = new uint[](count);
    for (uint i = 0; i < count; ) {
        ids[i] = i;
        unchecked { ++i; }  // safe: i < count
    }
    return ids;
}
```

Listing 4: Memory-computed `getAllCampaignIds()` – zero storage reads

Saving: ~27,100 gas per `createCampaign()` call. Additional benefit: `getAllCampaignIds()` is now a pure memory operation with no `SLOAD` cost, making it scale to any number of campaigns.

3.4. Other minor improvements

Variable ordering in `struct`:

The variables inside the `struct` are ordered intuitively in a descending order based on the amount of storage (in bytes) they would possibly take up to leverage the packing 32-byte slot usage, minimizing the `struct` storage take up and cost of the gas accordingly.

uint storage capacity control:

Variables like goalwei , deadline or any integer data can be stored as uint128 or uint96 instead of uint256, which would offer lesser but practically enough space for the variables saving gas.

Storage Caching:

Every `SLOAD` of storage takes 2100 gas(cold) or 700(gas) , though low multiple calls can add to up higher amount of gas leading to gas wastage. Caching the attribute for further usage to a local uint variable comparatively less gas per call.

4. Summary of All Changes

Table 3: Complete optimization summary

ID	Change	Gas saved / call	Section
OPT-1	Remove `uint campaignid` from struct	~22,100	3.1
OPT-2	Remove `string metadataCID` from struct & storage	~66,300	3.2
OPT-3	Remove `uint[] campaign_id_list`; compute in memory	~27,100	3.3
OPT-4	Remove redundant zero-value `SSTORE` assignments	~200	§4.1
OPT-5	`unchecked {++i}` in `getAllCampaignIds()`	~30/iter	§4.2
	Total `createCampaign()` saving (estimated)	~115,700	